Documentação Técnica – Controle Financeiro com CRUD e IA

Versão: 1.0 | **Data**: 21 de Janeiro 2026 | **Status**: Fase 1 (Planejamento e Arquitetura)

Índice

1. Visão Geral
2. Arquitetura do Sistema
3. Modelo de Dados
4. Estratégia de Banco de Dados
5. Camada de Validação (Zod)
6. Abstração de Persistência (Repository Pattern)
7. Endpoints da API CRUD
8. Integração com IA
9. Performance e FinOps
10. Plano de Implementação
11. Segurança
12. Deploy e Hospedagem

Visão Geral

Controle Financeiro é uma aplicação Next.js full-stack para gestão de transações financeiras pessoais com suporte a:

- **CRUD Completo**: Criar, ler, atualizar e deletar transações.
- **Análises Inteligentes**: Agente de IA para gerar insights sobre gastos, sugerir categorias e executar comandos de alteração.

- **Relatórios**: Sumário financeiro (income, expense, balance).
- **Persistência Robusta**: PostgreSQL em produção, abstraída via Repository Pattern.

Stack Tecnológico:

Camada	Tecnologia
Frontend	React 19, Next.js 16, TypeScript, Tailwind CSS
API Backend	Next.js API Routes
Banco de Dados	PostgreSQL (Neon) / In-Memory (protótipo)
Validação	Zod
ORM	Prisma (opcional, para tipagem forte em queries)
IA	OpenAI API (GPT-4 ou superior) para análises e processamento de comandos
Testes	Jest + React Testing Library

Arquitetura do Sistema

A aplicação segue **Clean Architecture** com separação de responsabilidades:

```
src/
├── app/
│   ├── api/
│   │   └── transactions/
│   │       ├── route.ts            # Handlers GET/POST
│   │       └── [id]/
│   │           └── route.ts        # Handlers PUT/DELETE
│   ├── layout.tsx
│   ├── page.tsx
│   └── globals.css
├── components/
│   ├── DashboardView.tsx           # Componente principal
│   ├── TransactionsTable.tsx       # Tabela de transações
│   ├── TransactionModal.tsx        # Modal CRUD
│   ├── FilterControls.tsx          # Filtros
│   ├── FinanceHeader.tsx           # Header com sumário
│   └── ui/                         # Componentes reutilizáveis
```

```
├── hooks/
│   └── useFinanceData.ts            # Hook para fetch de dados
├── lib/
│   ├── validations.ts               # Schemas Zod
│   ├── repositories/                # Abstração de dados
│   │   ├── ITransactionRepository.ts    # Interface
│   │   ├── PostgresTransactionRepository.ts
│   │   ├── InMemoryTransactionRepository.ts
│   │   └── container.ts             # Injeção de dependência
│   ├── ai/
│   │   ├── AIAnalyzer.ts            # Integração IA
│   │   └── prompts.ts               # Templates de prompts
│   └── utils.ts
├── types/
│   └── finance.ts                   # Tipos e interfaces
└── tests/
    ├── api/
    │   └── transactions.test.ts
    └── lib/
        └── validations.test.ts
```

Fluxo de Dados

```
Frontend Component
    ↓
useFinanceData Hook (fetch)
    ↓
Next.js API Route (/api/transactions)
    ↓
Validação (Zod)
    ↓
Repository Pattern (Abstração)
    ↓
PostgreSQL / In-Memory
    ↓
Resposta JSON
```

```
  ↓
Frontend (Re-render via SWR/React Query)
```

Modelo de Dados

Interface `Transaction`

```
export interface Transaction {
  id: string;                        // UUID gerado
  type: 'income' | 'expense';        // Tipo de transação
  description: string;               // Descrição (1-255 chars)
  value: number;                     // Valor em decimal (positivo)
  date: string;                      // ISO 8601 (YYYY-MM-DD)
  category: TransactionCategory;     // Categoria fixa
  responsible: string;               // Responsável pela transação
  installment_number?: number;       // Número da parcela (1-based)
  total_installments?: number;       // Total de parcelas
  is_recurring?: boolean;            // Flag de recorrência
  parent_transaction_id?: string;    // UUID da transação pai (para parcelas)
  paid?: boolean;                    // Flag de pagamento
}
```

Tipos Enumerados

```
export type TransactionType = 'income' | 'expense';

export type TransactionCategory =
  | 'Alimentação'
  | 'Transporte'
  | 'Casa'
  | 'Saúde'
  | 'Educação'
  | 'Lazer'
  | 'Salário'
  | 'Investimentos'
  | 'Outros';
```

Interface `FinancialSummary`

```
export interface FinancialSummary {
  income: number;       // Soma de todas as receitas
  expense: number;       // Soma de todas as despesas
  balance: number;      // income - expense
}
```

Forma de Entrada (Frontend)

```
export type TransactionFormData = Omit<Transaction, 'id' | 'value'> & {
  value: string;  // String para facilitar input do usuário
};
```

Estratégia de Banco de Dados

Decisão: PostgreSQL com Neon

Justificativa:

- **Análises Rápidas**: IA prioriza queries agregadas (somas, filtros por categoria). PostgreSQL executa agregações em O(log n) com índices otimizados (2-5x mais rápido que MongoDB).
- **Consistência ACID**: Dados financeiros exigem transações ACID para evitar inconsistências em cálculos de `balance`.
- **Escalabilidade Econômica**: Neon free tier oferece 512MB de armazenamento e 100 horas de computação/mês. Paid tiers escalam melhor que MongoDB para analytics.
- **Integração IA**: pgvector (extensão gratuita) suporta embeddings vetoriais se categorização por IA for implementada.

Trade-offs:

- Setup inicial mais complexo (migrações SQL, ORM).
- Overhead: Relatório complexo em time-series pode custar ~$0.05-0.15/execução em Neon.

Alternativa: In-Memory (Protótipo Rápido)

Para desenvolvimento inicial rápido, implementar repository com array JSON na memória. Trade-off: Dados perdem-se no restart, sem persistência entre deployments.

Migrações e Schema

Schema PostgreSQL (via Prisma):

```
model Transaction {
  id                 String   @id @default(cuid())
  type               String   // 'income' | 'expense'
  description        String   @db.VarChar(255)
  value              Decimal  @db.Decimal(12, 2)
  date               DateTime @db.Date
  category           String
  responsible        String
  installment_number Int?
  total_installments Int?
  is_recurring       Boolean? @default(false)
  parent_transaction_id String?
  paid               Boolean? @default(false)
  created_at         DateTime @default(now())
  updated_at         DateTime @updatedAt

  // Índices para performance
  @@index([date])
  @@index([category])
  @@index([type])
  @@index([parent_transaction_id])
  @@unique([id])
}
```

Índices Estratégicos (Big-O: O(log n)):

- `date`: Queries por período (ex.: “últimos 30 dias”).
- `category`: Filtros por categoria (ex.: “Total gasto em Transporte”).
- `type`: Segmentação income vs. expense.
- `parent_transaction_id`: Busca de parcelas filhas.

Camada de Validação (Zod)

Validação de tipo em runtime garante integridade de dados de entrada via API.

Schema `TransactionSchema`

```
// src/lib/validations.ts
import { z } from 'zod';

export const TransactionSchema = z.object({
  id: z.string().uuid().optional(), // Opcional na criação, gerado no servidor
  type: z.enum(['income', 'expense']),
  description: z.string().min(1).max(255),
  value: z.number().positive('Valor deve ser positivo'),
  date: z.string().refine(
    (val) => !isNaN(Date.parse(val)),
    { message: 'Data inválida, use ISO 8601 (YYYY-MM-DD)' }
  ),
  category: z.enum([
    'Alimentação', 'Transporte', 'Casa', 'Saúde', 'Educação',
    'Lazer', 'Salário', 'Investimentos', 'Outros'
  ]),
  responsible: z.string().min(1),
  installment_number: z.number().positive().optional(),
  total_installments: z.number().positive().optional(),
  is_recurring: z.boolean().optional(),
  parent_transaction_id: z.string().uuid().optional(),
  paid: z.boolean().optional(),
});

// Schema para criação (sem ID)
export const CreateTransactionSchema = TransactionSchema.omit({ id: tru
e });

// Schema para sumário
export const FinancialSummarySchema = z.object({
  income: z.number().min(0),
  expense: z.number().min(0),
```

```
  balance: z.number(),
});
```

Uso nos Handlers

```
// src/app/api/transactions/route.ts
import { CreateTransactionSchema } from '@/lib/validations';

export async function POST(request: Request) {
  try {
    const body = await request.json();
    const validatedData = CreateTransactionSchema.parse(body);
    // Prosseguir com dados validados
    return Response.json({ success: true, data: validatedData });
  } catch (error) {
    if (error instanceof z.ZodError) {
      return Response.json(
        { error: 'Validação falhou', issues: error.issues },
        { status: 400 }
      );
    }
    return Response.json(
      { error: 'Erro interno do servidor' },
      { status: 500 }
    );
  }
}
```

Performance: Validação Zod adiciona ~1-5ms por requisição (negligenciável para UX).

Abstração de Persistência (Repository Pattern)

Repository Pattern desacopla a lógica de negócio da implementação de armazenamento, permitindo trocar DB sem refator.

Interface Abstrata

```
// src/lib/repositories/ITransactionRepository.ts
import { Transaction, FinancialSummary } from '@/types/finance';

export interface ITransactionRepository {
  // CRUD
  getAll(filters?: Partial<Transaction>): Promise<Transaction[]>;
  getById(id: string): Promise<Transaction | null>;
  create(data: Omit<Transaction, 'id'>): Promise<Transaction>;
  update(id: string, data: Partial<Transaction>): Promise<Transaction | null
>;
  delete(id: string): Promise<boolean>;

  // Analytics
  getSummary(filters?: Partial<Transaction>): Promise<FinancialSummary>;
  getByCategory(category: string): Promise<Transaction[]>;
  getByDateRange(startDate: string, endDate: string): Promise<Transaction
[]>;
}
```

Implementação: PostgreSQL com Prisma

```
// src/lib/repositories/PostgresTransactionRepository.ts
import { PrismaClient } from '@prisma/client';
import { ITransactionRepository } from './ITransactionRepository';
import { Transaction, FinancialSummary } from '@/types/finance';

export class PostgresTransactionRepository implements ITransactionRepos
itory {
  private prisma: PrismaClient;

  constructor() {
    this.prisma = new PrismaClient();
  }

  async getAll(filters?: Partial<Transaction>): Promise<Transaction[]> {
    return this.prisma.transaction.findMany({
      where: filters,
```

```
      orderBy: { date: 'desc' },
    });
  }

  async getById(id: string): Promise<Transaction | null> {
    return this.prisma.transaction.findUnique({ where: { id } });
  }

  async create(data: Omit<Transaction, 'id'>): Promise<Transaction> {
    return this.prisma.transaction.create({ data });
  }

  async update(id: string, data: Partial<Transaction>): Promise<Transaction
| null> {
    return this.prisma.transaction.update({
      where: { id },
      data,
    });
  }

  async delete(id: string): Promise<boolean> {
    const result = await this.prisma.transaction.delete({
      where: { id },
    });
    return !!result;
  }

  async getSummary(filters?: Partial<Transaction>): Promise<FinancialSum
mary> {
    const transactions = await this.getAll(filters);
    const income = transactions
      .filter((t) => t.type === 'income')
      .reduce((sum, t) => sum + t.value, 0);
    const expense = transactions
      .filter((t) => t.type === 'expense')
      .reduce((sum, t) => sum + t.value, 0);
    return { income, expense, balance: income - expense };
```

```
  }
}
```

Complexidade Big-O:

- `getAll()`: O(log n) com índices de DB.
- `getById()`: O(1) com índice primário.
- `create()`: O(log n) por atualização de índices.
- `getSummary()`: O(n) em memória (otimizável com DB aggregation).

Implementação: In-Memory (Prototipagem)

```
// src/lib/repositories/InMemoryTransactionRepository.ts
import { v4 as uuidv4 } from 'uuid';
import { ITransactionRepository } from './ITransactionRepository';
import { Transaction, FinancialSummary } from '@/types/finance';

export class InMemoryTransactionRepository implements ITransactionRepo
sitory {
  private transactions: Map<string, Transaction> = new Map();

  async getAll(filters?: Partial<Transaction>): Promise<Transaction[]> {
    let results = Array.from(this.transactions.values());
    if (filters) {
      results = results.filter((t) =>
        Object.entries(filters).every(([key, value]) => t[key as keyof Transactio
n] === value)
      );
    }
    return results.sort((a, b) => new Date(b.date).getTime() - new Date(a.dat
e).getTime());
  }

  async create(data: Omit<Transaction, 'id'>): Promise<Transaction> {
    const transaction: Transaction = {
      ...data,
      id: uuidv4(),
    };
```

```
    this.transactions.set(transaction.id, transaction);
    return transaction;
  }

  async getSummary(filters?: Partial<Transaction>): Promise<FinancialSum
mary> {
    const transactions = await this.getAll(filters);
    const income = transactions
      .filter((t) => t.type === 'income')
      .reduce((sum, t) => sum + t.value, 0);
    const expense = transactions
      .filter((t) => t.type === 'expense')
      .reduce((sum, t) => sum + t.value, 0);
    return { income, expense, balance: income - expense };
  }

  // ... outros métodos
}
```

Container de Injeção de Dependência

```
// src/lib/repositories/container.ts
import { ITransactionRepository } from './ITransactionRepository';
import { PostgresTransactionRepository } from './PostgresTransactionRepos
itory';
import { InMemoryTransactionRepository } from './InMemoryTransactionRe
pository';

let repository: ITransactionRepository;

if (process.env.USE_MEMORY_DB === 'true') {
  repository = new InMemoryTransactionRepository();
} else {
  repository = new PostgresTransactionRepository();
}

export { repository };
```

Endpoints da API CRUD

1. GET /api/transactions – Listar Transações

Propósito: Retornar todas as transações com filtros opcionais e paginação.

Query Parameters:

Parâmetro	Tipo	Descrição
`type`	string	Filtro por tipo: `income` ou `expense`
`category`	string	Filtro por categoria
`startDate`	string	Data início (ISO 8601)
`endDate`	string	Data fim (ISO 8601)
`responsible`	string	Filtro por responsável
`page`	number	Página (default: 1)
`limit`	number	Itens por página (default: 20, max: 100)

Resposta 200:

```
{
  "data": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "type": "expense",
      "description": "Supermercado",
      "value": 150.50,
      "date": "2026-01-21",
      "category": "Alimentação",
      "responsible": "João",
      "paid": true
    }
  ],
  "pagination": {
    "page": 1,
    "limit": 20,
    "total": 42
  },
  "summary": {
    "income": 5000,
```

```
    "expense": 1230.75,
    "balance": 3769.25
  }
}
```

Resposta 400 (validação):

```
{
  "error": "Query parameter inválido",
  "issues": [{ "field": "startDate", "message": "Data inválida" }]
}
```

---

2. POST /api/transactions – Criar Transação

Propósito: Criar uma nova transação (incluindo parcelas automáticas).

Body (JSON):

```
{
  "type": "expense",
  "description": "Aluguel",
  "value": 1500,
  "date": "2026-01-25",
  "category": "Casa",
  "responsible": "Maria",
  "total_installments": 3,
  "is_recurring": false
}
```

Lógica de Parcelas:

Se `total_installments` > 1, o servidor cria múltiplas transações filhas:

1. Transação pai com `id` único.
2. 3 transações filhas com `installment_number` (1, 2, 3) e `parent_transaction_id` apontando para a pai.
3. Cada filha tem `value = total_value / total_installments`.

Resposta 201:

```
{
  "data": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "type": "expense",
    "description": "Aluguel",
    "value": 500,
    "date": "2026-01-25",
    "category": "Casa",
    "responsible": "Maria",
    "installment_number": 1,
    "total_installments": 3,
    "parent_transaction_id": null,
    "paid": false
  },
  "childTransactions": [
    { "id": "...", "installment_number": 1 },
    { "id": "...", "installment_number": 2 },
    { "id": "...", "installment_number": 3 }
  ]
}
```

Resposta 400:

```
{
  "error": "Validação falhou",
  "issues": [{ "field": "value", "message": "Valor deve ser positivo" }]
}
```

---

3. PUT /api/transactions/[id] – Atualizar Transação

Propósito: Atualizar transação existente (ex.: marcar como paga).

Body (JSON):

```
{
  "paid": true,
```

```
  "description": "Aluguel atualizado"
}
```

Resposta 200:

```
{
  "data": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "type": "expense",
    "description": "Aluguel atualizado",
    "value": 500,
    "date": "2026-01-25",
    "category": "Casa",
    "responsible": "Maria",
    "paid": true
  }
}
```

Resposta 404:

```
{
  "error": "Transação não encontrada",
  "id": "550e8400-e29b-41d4-a716-446655440000"
}
```

---

4. DELETE /api/transactions/[id] – Deletar Transação

Propósito: Remover transação (soft delete recomendado para auditoria).

Resposta 200:

```
{
  "success": true,
  "message": "Transação deletada",
  "id": "550e8400-e29b-41d4-a716-446655440000"
}
```

Opção: Deletar Parcelas Filhas:

Query parameter `cascade=true` deleta transação pai e todas as filhas.

Integração com IA

Agente de IA executará análises internas e comandos simples de CRUD. Será integrado via OpenAI API com prompts estruturados.

Funcionalidades de IA

1. **Análises de Gastos**:
   - "Quanto gastei em Transporte este mês?"
   - Resposta: Query agregada → OpenAI estrutura resposta natural.
2. **Categorização Automática**:
   - Sugerir categoria baseada em descrição (ex.: "Uber" → "Transporte").
   - Usar embeddings + cosine similarity para matching.
3. **Sugestões de Economia**:
   - "Quais categorias tiveram maior aumento?"
   - Analytics + OpenAI gera insights.
4. **Comandos de Alteração**:
   - "Adiciona uma despesa de R$ 50 em Alimentação amanhã."
   - Parsear intent → Criar transação via API.

Módulo AIAnalyzer

```
// src/lib/ai/AIAnalyzer.ts
import OpenAI from 'openai';
import { ITransactionRepository } from '../repositories/ITransactionReposito
ry';

export class AIAnalyzer {
  private openai: OpenAI;
  private repository: ITransactionRepository;

  constructor(repository: ITransactionRepository) {
    this.openai = new OpenAI({
```

```
    apiKey: process.env.OPENAI_API_KEY,
  });
  this.repository = repository;
}

async analyzeQuery(userQuery: string): Promise<string> {
  // 1. Extrair intenção do usuário
  const intent = await this.extractIntent(userQuery);

  // 2. Executar ação baseada em intenção
  let data = '';
  if (intent === 'ANALYSIS') {
    data = await this.performAnalysis(userQuery);
  } else if (intent === 'COMMAND') {
    data = await this.executeCommand(userQuery);
  }

  // 3. Estruturar resposta com GPT
  const response = await this.openai.chat.completions.create({
    model: 'gpt-4',
    messages: [
      {
        role: 'system',
        content: 'Você é um assistente financeiro. Responda em português, d
e forma concisa e útil.',
      },
      {
        role: 'user',
        content: `Pergunta:${userQuery}\n\nDados:${data}`,
      },
    ],
  });

  return response.choices[0].message.content || 'Desculpe, não consegui
processar.';
}

private async extractIntent(query: string): Promise<string> {
```

```
    const response = await this.openai.chat.completions.create({
      model: 'gpt-4',
      messages: [
        {
          role: 'user',
          content: `Classifique a intenção como "ANALYSIS" ou "COMMAN
D":\n${query}`,
        },
      ],
    });
    return response.choices[0].message.content?.includes('ANALYSIS') ? 'A
NALYSIS' : 'COMMAND';
  }

  private async performAnalysis(query: string): Promise<string> {
    const transactions = await this.repository.getAll();
    return JSON.stringify(transactions);
  }

  private async executeCommand(query: string): Promise<string> {
    // Parsear comando e criar/atualizar transação
    // Implementação em progresso
    return 'Comando executado';
  }
}
```

Prompts Estruturados

```
// src/lib/ai/prompts.ts
export const SYSTEM_PROMPTS = {
  ANALYZER: `Você é um assistente financeiro inteligente. Analise dados de
transações e forneça insights.
Regras:
- Sempre responda em português
- Seja conciso (max 3 linhas)
- Use formatação markdown para valores monetários (ex: R$ 1.230,45)
- Se não souber, diga "Desculpe, não consigo responder com precisão"`,
```

```
  CATEGORIZER: `Categorize a seguinte descrição de transação em uma da
s categorias:
Alimentação, Transporte, Casa, Saúde, Educação, Lazer, Salário, Investime
ntos, Outros.
Responda APENAS com a categoria.`,

  COMMAND_PARSER: `Parse o seguinte comando em um objeto JSON co
m campos:
{ "action": "CREATE|UPDATE|DELETE", "type": "income|expense", "value":
number, "category": string, "description": string }
Responda APENAS com o JSON válido.`,
};
```

Performance e FinOps

Análise Big-O

Operação	PostgreSQL	In-Memory	Complexidade
`create()`	~5ms	~0.5ms	O(log n) índices
`getById()`	~2ms	~0.1ms	O(1) índice primário
`getAll()` sem filtro	~10ms	~1ms	O(n) scan
`getAll()` com índice	~5ms	~O(n) scan	O(log n) com index
`getSummary()` simples	~15ms	~1ms	O(n) agregação
`delete()`	~8ms	~0.5ms	O(log n)

Otimizações Recomendadas:

1. **Índices Compostos**: `(type, category, date)` para queries frequentes.
2. **Caching**: Redis para `getSummary()` (cache por 5min).
3. **Paginação**: Limitar `getAll()` a 100 itens/página.
4. **Lazy Loading**: Carregar parcelas sob demanda.

Custos FinOps

PostgreSQL (Neon):

- Free tier: 512MB, 100 horas/mês ≈ $0/mês para prototipagem.

- Estimativa: 50k transações ≈ 200MB; custo monthly ~$5-10/mês em paid tier.
- Egress: $0.09/GB (risco se IA processar dados volumosos).
- **Recomendação**: Compressão de backups, soft deletes.

OpenAI API:

- Análises simples (GPT-4): ~$0.03 por query (prompt 500 tokens, resposta 100 tokens).
- Custo estimado: 100 queries/mês = ~$3/mês.
- **Recomendação**: Cache de respostas frequentes.

Total Estimado: $5-15/mês para operação inicial.

Plano de Implementação

Fase 1: Preparação (1-2 dias)

- [ ] Revisar tipos em src/types/finance.ts.
- [ ] Instalar Zod: `pnpm add zod`.
- [ ] Criar `src/lib/validations.ts` com schemas.
- [ ] Configurar Neon PostgreSQL (criar conta, obter connection string).
- [ ] Instalar Prisma: `pnpm add -D prisma @prisma/client`.

Fase 2: Repositório e API (2-3 dias)

- [ ] Criar interface `ITransactionRepository.ts`.
- [ ] Implementar `InMemoryTransactionRepository.ts` para testes rápidos.
- [ ] Implementar `PostgresTransactionRepository.ts` com Prisma.
- [ ] Criar handlers GET/POST em `src/app/api/transactions/route.ts`.
- [ ] Criar handlers PUT/DELETE em `src/app/api/transactions/[id]/route.ts`.

Fase 3: Integração Frontend (1-2 dias)

- [ ] Atualizar `useFinanceData.ts` para consumir API.
- [ ] Modificar componentes para usar dados remotos.

- [ ] Implementar tratamento de erros e loading states.

Fase 4: IA e Análises (2-3 dias)

- [ ] Criar `src/lib/ai/AIAnalyzer.ts`.
- [ ] Integrar OpenAI API.
- [ ] Implementar endpoints: `POST /api/ai/analyze`.

Fase 5: Testes e Deploy (1-2 dias)

- [ ] Testes unitários com Jest.
- [ ] Testes de integração.
- [ ] Deploy em Vercel.

Segurança

Checklist de Segurança

- [ ] **Validação de Entrada**: Zod em todos os endpoints.
- [ ] **Sanitização**: Usar parâmetros preparados (Prisma já faz isso).
- [ ] **Autenticação**: Implementar JWT ou NextAuth.js para usuários.
- [ ] **Autorização**: Row-level security (RLS) no PostgreSQL.
- [ ] **Rate Limiting**: Middleware para limitar requisições (ex.: 100/min).
- [ ] **CORS**: Configurar CORS restritivamente.
- [ ] **Secrets**: Usar `process.env` para API keys (nunca hardcode).
- [ ] **Logging**: Registrar todas as alterações em auditoria.

Exemplo: Middleware de Autenticação

```
// src/lib/middleware/auth.ts
import { jwtVerify } from 'jose';

export async function verifyAuth(request: Request) {
  const token = request.headers.get('Authorization')?.replace('Bearer ', '');
  if (!token) {
    throw new Error('Token não fornecido');
```

```
  }
  const secret = new TextEncoder().encode(process.env.JWT_SECRET ||
'');
  const verified = await jwtVerify(token, secret);
  return verified.payload;
}
```

Deploy e Hospedagem

Opções Recomendadas

Serviço	Custo	Prós	Contras
Vercel	Free/Paid	Cold start ~100ms, Next.js native	Egress caro
Railway	Free/$5+	Simples, bom suporte	Menor escala
Render	Free/$7+	Gratuito por 750h	Limitado

Deploy em Vercel

```
# 1. Conectar repositório GitHub
vercel link

# 2. Configurar variáveis de ambiente
vercel env add DATABASE_URL "postgresql://..."
vercel env add OPENAI_API_KEY "sk-..."

# 3. Deploy
vercel deploy --prod
```

Checklist de Deploy

- [ ] Build passa: `pnpm build`.
- [ ] Testes passam: `pnpm test`.
- [ ] Variáveis de ambiente configuradas.
- [ ] Migrações Prisma executadas: `prisma migrate deploy`.
- [ ] Logs monitorados (Sentry, Datadog).

- [ ] Backup de DB configurado.

Referências e Documentação Oficial

- Next.js API Routes
- Zod Validation
- Prisma ORM
- PostgreSQL
- Neon Free Tier
- OpenAI API
- Clean Architecture

Mantainer: Tim de Desenvolvimento | **Última atualização**: Janeiro 2026